

Vector Databases: A Primer

A vector database stores high-dimensional vectors and supports fast similarity search over them. Instead of exact keyword matching, retrieval is based on distance between embeddings, which lets systems find semantically related items even when they share no common terms.

Embeddings are produced by machine learning models that map objects such as text, images, or audio into a vector space where similar items are close together. For example, the sentence "cats are pets" and "felines are domesticated animals" would be placed near each other despite using completely different words.

A typical RAG pipeline first indexes a corpus by embedding every chunk of text and storing the vectors, then answers a question by embedding the query and searching for the most similar stored vectors. The retrieved passages are finally handed to a language model as context for generating the answer.

Example: Python snippet

The code below demonstrates a minimal similarity search with numpy:

```
` ``python
import numpy as np

def cosine_similarity(a, b):
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

def search(query_vec, index, top_k=3):
    scores = [(cosine_similarity(query_vec, v), i) for i, v in enumerate(index)]
    scores.sort(reverse=True)
    return [i for _, i in scores[:top_k]]

vectors = np.random.rand(10, 4)
q = np.random.rand(4)
print(search(q, vectors))
` ``
```

Cosine similarity ignores the magnitude of the vectors and only measures the angle between them, which makes it a popular metric for text embeddings that are normalized before storage.

Unicode and special characters

Technical documents frequently contain mathematical symbols and arrows: the Greek letters lambda (λ) and pi (π), the arrow ($\rightarrow$), plus common operators such as $\pm$, $\times$, and $\div$. A robust text pipeline must preserve these characters so that chunking and embedding see the original content, not replacement glyphs.

This page exists mainly to verify that the PDF fixture round-trips these characters through PyMuPDF without encoding corruption, which would otherwise silently degrade retrieval quality.